45 DAY FAANG INTERVIEW PREPARATION PLAN

Complete Study Schedule

DSA PLAN (DAYS 1 - 30)

Day	DSA Topic	NC Problems
1-3	Arrays & Hashing I, II, III	NC 1 - 9
4-5	Two Pointers I, II	NC 10 - 15
6	Revision & Easy Practice	Revision
7-9	Sliding Window I, II, III	NC 16 - 24
10-12	Stack I, II, Monotonic Stack	NC 25 - 33
13	Queue / Deque	NC 34 - 36
14-15	Binary Search I, II	NC 37 - 42
16-18	Linked List I, II, III	NC 43 - 51
19-22	Trees I, II, III, IV	NC 52 - 63
23-26	BFS I & II, DFS I & II	NC 64 - 75
27	Graphs (Algos)	NC 76 - 78
28	Heap / Priority Queue	NC 79 - 81
29-30	Backtracking I, II	NC 82 - 87

DSA PLAN (DAYS 31 - 45) + TIMETABLE

Day	DSA Topic	NC Problems
31	Backtracking III	NC 88 - 90
32-33	Intervals I, II	NC 91 - 96
34-35	Greedy I, II	NC 97 - 102
36-38	Dynamic Programming I, II, III	NC 103 - 111
39	DP (Advanced / Knapsack / LCS)	NC 112 - 114
40-42	Mixed Practice I, II, III	NC 115 - 123
43-44	Contest / Hard Problems	NC 124 - 129
45	Final Revision & Mock Interviews	Review

DAILY STUDY TIMETABLE:

- **8:30 - 10:30 PM: Solve 2 Problems (P1 + P2)**
- **10:30 - 11:00 PM: Dry Run + Complexity + Explain**
- **12:00 - 12:45 AM: Solve 3rd Problem (P3)**
- **12:45 - 1:00 AM: Behavioral Topic Practice**
- **1:00 - 1:30 AM: LLD / HLD Concept Practice**

BEHAVIORAL & SYSTEM DESIGN PLAN

Day	Behavioral / Leadership	LLD / HLD Topic
1-5	Intro, Achievement, Failure, Conflict	SOLID, URL Shortener, Factory
6-10	Leadership, Mentoring, Influence	Rate Limiter, Strategy, Observer
11-15	Customer Focus, Innovation, Priority	Chat System, Singleton, News Feed
16-20	Teamwork, Pressure, Risk Taking	Decorator, Builder, Search Engine
21-25	Prioritization, RCA, Debugging	Payment System, Proxy, Ride Sharing
26-30	Architecture, Trade-offs, Quality	Command, Rate Limiter 2.0, Composite
31-35	Promotion, Manager Rel., Negotiation	State, Distributed Cache, Bridge
36-40	Hiring, Metric Driven, Security	Mediator, Analytics Pipeline, Memento
41-45	Design Thinking, Challenge Story	Visitor, Real-time System, Mock

VOLUME 1: ARRAYS & HASHING

1. INTERVIEW FLOW (ALWAYS FOLLOW)

1. **Clarify:** Ask constraints, edge cases, duplicates, range, complexity.
2. **Brute Force:** Get it working first (O(N^2)).
3. **Better:** Can we optimize? Using HashMap/Set.
4. **Optimal:** Best approach in Time/Space (O(N) or O(1)).
5. **Code:** Clean, modular, bug-free code.
6. **Explain:** Walkthrough + dry run.

2. PATTERN RECOGNITION

Need	Use	Example Problems
Uniqueness / Duplicate	HashSet	Contains Duplicate (217)
Frequency / Count	HashMap / int[26]	Valid Anagram (242), Top K (347)
Pair / Complement	HashMap	Two Sum (1)
Grouping	HashMap<Key, List>	Group Anagrams (49)
Top K Frequent	HashMap + Min Heap	Top K Frequent Elements (347)
Left & Right Info	Prefix / Suffix array	Product of Array Except Self (238)

3. PROBLEMS AT A GLANCE

LC #	Problem	Pattern	Key Idea
217	Contains Duplicate	HashSet	If !set.add(x) ⇒ duplicate
242	Valid Anagram	Freq Array	Count++ then Count--
1	Two Sum	HashMap	Find target - num in map
49	Group Anagrams	HashMap	Use 26-count signature key
347	Top K Frequent	HashMap+Heap	Min Heap of size K
271	Encode & Decode	Delimiter	length#string format
238	Product Except Self	Prefix/Suffix	Left[i] * Right[i]

Next Page: Arrays & Hashing Volume 2

4. KEY PATTERNS & TRICKS (HIGH IMPACT)

Group Anagrams (LC 49):

```
int[] freq = new int[26];
for(char c : s.toCharArray()) freq[c - 'a']++;
StringBuilder key = new StringBuilder();
for(int f : freq) key.append(f).append('#');
```

Top K Frequent (LC 347):

```
PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);
// Keep min heap of size K to maintain Top K elements
```

5. COMPLEXITY QUICK REFERENCE

Operation	Time Complexity
HashSet add / contains / remove	O(1) average
HashMap put / get / contains	O(1) average
Sort an array of size N	O(N log N)
PriorityQueue offer / poll	O(log K) for size K

6. JAVA COLLECTIONS & APIS (MOST USED)

HashSet: add(x), contains(x), remove(x), size(), clear()
HashMap: put(k,v), get(k), containsKey(k), getOrDefault(k, def), computeIfAbsent(k, v->new)
PriorityQueue: offer(x), poll(), peek(), size()
Conversions:

- String to char[]: s.toCharArray()
- char[] to String: new String(arr)
- String to int: Integer.parseInt(s)
- char to int: c - '0'
- List to array: list.toArray(new T[0])

9. COMMON MISTAKES (AVOID)

- ❌ Using sorting when a HashSet can solve it in O(N).
- ❌ Forgetting to check if num-1 exists in set before starting sequence counting (Longest Consecutive Sequence).
- ❌ Using simple String.split() in Encode/Decode (fails with consecutive delimiters).
- ❌ Not handling negative numbers or zero edge cases in hash functions or array indexing.
- ❌ Assuming HashSet preserves element insertion order (use LinkedHashSet if order is needed).

10. INTERVIEW COMMUNICATION TIPS & MANTRAS

Communication Tips:

- Think out loud. Explain your choice of HashSet/HashMap before writing code.
- Start with the brute force, state its complexity, and then introduce the optimal O(N) map-based approach.
- Write clean, readable variable names (e.g. counts instead of).

Memory Mantras:

- Need Uniqueness? ⇒ Use **HashSet**
- Need Count / Freq? ⇒ Use **HashMap**
- Need Left/Right Info? ⇒ Use **Prefix/Suffix product**
- Need Top-K? ⇒ Use **Heap / Priority Queue**